BPC Project: Scaling Laws of Cardinality Estimators

1. Overview

The project has two phases:

1. **Build `kgsynth`**, a procedural synthetic knowledge-graph generator whose output statistically resembles real RDF KGs (DBpedia, Wikidata, SWDF, FB15k-237) but generalizes them along controllable axes. The package is **general-purpose**, not FICE-specific.
2. **Run a scaling-law study** of FICE: train on increasing numbers of `kgsynth` graphs, evaluate on held-out synthetic graphs *and* real KGs, fit empirical scaling curves.

Why this matters

FICE is fully inductive — it generalizes to graphs and relations not seen during training. If the scaling curve is favorable and synthetic-only training transfers to real KGs, we get a recipe for training a strong, general-purpose cardinality estimator without the data-collection bottleneck of real KGs.

What "success" looks like

- A clean `kgsynth` package that, given a target signature, produces graphs whose measured signature closely matches the target across a defined set of features.
- Empirical scaling curves `Q-error(N)` for several test sets, with fitted power-law exponents.

---

2. Background

2.1 RDF cardinality estimation

An RDF knowledge graph is a set of triples `(s, p, o)` (subject, predicate, object). A SPARQL Basic Graph Pattern (BGP) is a small graph of triple patterns where some positions are variables, e.g.

```
?x  livesIn   ?city .
?x  worksAt   ?company .
?company  locatedIn  ?city .
```

The **cardinality** of the BGP is the number of variable assignments that satisfy it. Estimating cardinality without executing the query is critical for query planning. FICE predicts `log(cardinality)` from the BGP plus the underlying KG.

2.2 What "fully inductive" means here

FICE has no fixed entity- or relation-specific embeddings, instead they are dynamically generated by an embedding GNN. At inference, it ingests an arbitrary KG and BGP and produces a cardinality based on the stored embeddings from the embedding GNN. Training is over a *population* of (graph, query) pairs. This is what makes a scaling-laws study with synthetic graphs sensible — the model is trained on many KGs, not one.

2.3 What a "scaling law" is

In the sense of Hestness et al. (2017) and Kaplan et al. (2020): an empirical curve relating prediction error to training-data quantity, typically of the form

```
error(N) = a · N^{−b} + c
```

with fitted exponent `b` and irreducible-error asymptote `c`. We will fit this with `N` = number of training graphs, and separately with `N` = total triples in training data.

---

3. Phase 1: Synthetic Knowledge Graph Generator

The output of phase 1 is the `kgsynth` package and a set of measured signatures of real KGs.

3.1 The graph signature

Every real KG and every generated KG is summarized by a **signature vector** `σ(G)` made of six blocks. Your goal is to implement:

- (a) measurement of every block on real KGs,
- (b) targeting of every block in `kgsynth`,
- (c) reporting of the per-block distance between generated and target signatures.

This section explains every feature in detail because the concepts are not standard CS curriculum.

Block A — Size and density

These are the simplest knobs.

Feature	Definition	Why it matters
`|V|`	distinct entities (subjects ∪ objects, excluding literals),i.e. number of nodes	overall scale
`|E|`	number of triples	overall scale
`|R|`	distinct relations (predicates)	schema width
density	`|E| / |V|^2`	how connected the graph is;
triples per entity	`|E| / |V|`	average edges per node
relation reuse	`|E| / |R|`	how often the average relation is used

Block B — Degree structure

This is the **dominant signal** for triple-pattern selectivity. If the query is `?x livesIn berlin`, the answer count is exactly the in-degree of `berlin` under `livesIn`.

For each entity `v`:

- **out-degree** `d_out(v)` = triples with `v` as subject
- **in-degree** `d_in(v)` = triples with `v` as object

Real KGs have **heavy-tailed** degree distributions — a few hub entities (`USA`, `Albert_Einstein`) appear in a huge fraction of triples; most entities appear in very few. We fit a **power law** `P(d) ∝ d^{−α}` using the `powerlaw` Python package, which also provides a Kolmogorov–Smirnov goodness-of-fit test against alternatives (lognormal, exponential, truncated power law).

Per-relation refinements (much more informative than aggregate degree):

Feature	Definition
object-multiplicity of r	for each subject `s`, count of objects `o` such that `(s, r, o) ∈ G`; report distribution + Zipf fit
subject-multiplicity of r	symmetric for objects
functionality of r	fraction of subjects with object-multiplicity = 1 (e.g. `bornIn`)
inverse-functionality of r	fraction of objects with subject-multiplicity = 1

Functional and inverse-functional relations behave very differently from many-to-many ones (`type` , `friend`); joins involving them have very different selectivities.

Block C — Schema and relation correlation

Relations don't appear randomly; they cluster. An entity that has `bornIn` very likely also has `nationality` . We measure this with the **relation co-occurrence matrix** `M` where

$$M[i, j] = | \{ s : \exists o_1\ o_2.\ (s, r_i, o_1) \in G \wedge (s, r_j, o_2) \in G \} |$$

Symmetric variant for objects. `M` is summarized by:

- top-k singular values (`k = 10`)
- density (fraction of nonzeros)
- entropy of each row's distribution

Because the project includes types (`rdf:type`), we additionally measure:

- number of classes `|T|`
- class-size distribution and its Zipf exponent
- **type–relation conditional** `P(r | type(s))` — given a subject's type, what relations does it use?

Block D — Characteristic sets (the most predictive feature for star queries)

Characteristic sets, introduced by Neumann & Moerkotte (2011), are the single strongest signal for star-query cardinality estimation. **Definition**:

$$CS(s) = \{ p : \exists o.\ (s, p, o) \in G \}$$

i.e. the set of relations on which entity `s` has at least one outgoing edge. If many entities share the same characteristic set, a star query asking for "entities with relations p_1, p_2, p_3" reduces to counting entities whose CS is a superset of $\{p_1, p_2, p_3\}$ — exactly the cardinality the estimator must predict.

Features:

Feature	Definition
`|CS|`	number of distinct characteristic sets
CS-frequency Zipf exponent	how skewed the popularity of CSs is (typically heavy-tailed)

Feature	Definition
CS-size distribution	sizes of the sets themselves (mean, median, p90)
per-CS cardinality vector	for each `(CS, r ∈ CS)`, mean and p90 object-multiplicity; aggregated per CS

Inverse characteristic sets `CS⁻¹(o) = { p : ∃s. (s, p, o) ∈ G }` for object-side stars — same features, symmetric.

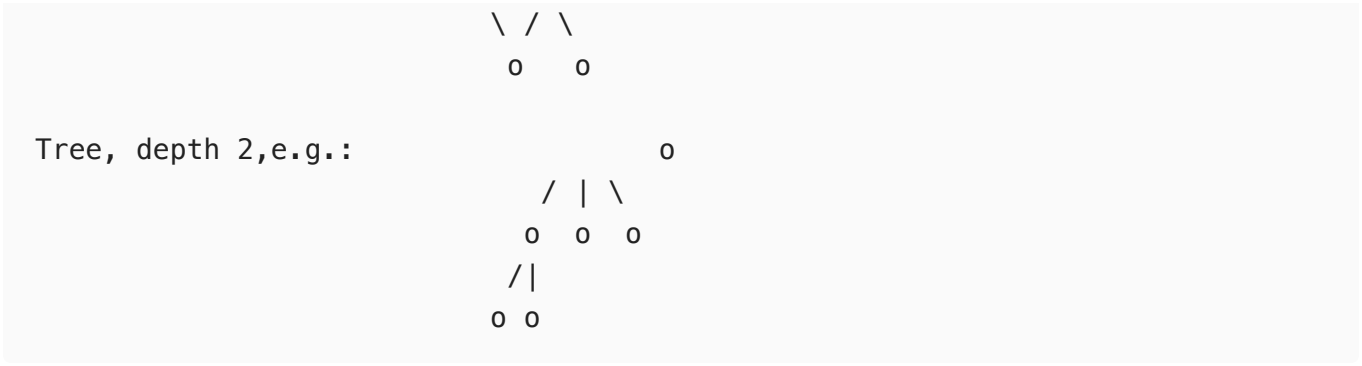
Two-step characteristic sets / characteristic pairs: pairs `(p, q)` such that `p` is outgoing and `q` incoming on a shared entity, or such that there is a path `s → p → x → q → o`. These predict path-2 selectivity; report top-k pair frequencies and the Zipf of the pair distribution.

Block E — Motifs (controllable shape distribution)

A **motif** is a small connected subgraph pattern. Different BGP query shapes correspond to different motifs in the KG, so the motif distribution determines query selectivity.

The motifs you should measure and target:

```
Path of length k:        o — o — o — ... — o    (k edges)

Star of degree k:              o
                               |
                          o —— o —— o          (one center, k pendants)
                               |
                               o

Triangle (3-cycle):       o — o
                           \ /
                            o

4-cycle:                  o — o
                          |   |
                          o — o

Diamond (K4 − edge):      o — o
                          | \ |          (one diagonal present,
                          o — o           one absent)

K4 (4-clique):            o — o
                          | × |          (both diagonals present)
                          o — o

Tailed triangle:          o — o
```



For BGP queries up to ~8 triple patterns, we care about:

- length-`k` **paths** for `k = 2..10`
- **stars** of degree `k = 2..10` (already fixed by characteristic sets)
- **k-cycles** for `k = 3..6`
- **diamonds** and **K4** (4-cliques)
- **rooted trees** of depth 2 and 3

For each, count occurrences in the graph. Exact counts are tractable for 3- and 4-node motifs; for 5- and 6-node motifs we use **sampling-based estimation** (e.g. Bressan et al. 2018 or Wanderjoin via G-Care): sample induced k-node subgraphs uniformly, classify via canonical labelling, estimate proportions with bounded standard error.

Path templates add the relation labels: a path template of length `k` is a directed walk's relation sequence `(r₁, r₂, …, r_k)`. We report:

- the Zipf exponent over template frequencies for each `k`
- the entropy of the template distribution

Tree templates are the analog for rooted depth-2 trees.

Block F — Connectivity

Feature	Definition
#components	number of weakly connected components
largest-component fraction	usually ≈ 1.0 in real KGs
avg shortest-path length	sampled over 10^4 pairs in the largest component
clustering coefficient	average local clustering
degree assortativity	Pearson correlation of endpoint degrees over edges; distinguishes hub-leaf from hub-hub graphs

3.2 Sufficiency vs exhaustiveness — be honest

Note that 2 graphs can have similar characteristics but still be different. The idea is to have graphs that are
sufficiently similar to real ones.

3.3 Measurement on real KGs

Targets: **SWDF, FB15k-237, DBpedia (a subset), Wikidata (a subset), YAGO (optional)**.

Pipeline:

1. Load the KG into an in-memory object (decide what is the most efficient representation here, torch, networkx etc. or selfbuild).
2. Compute every feature in §3.1. For sampling-based features (5+ node motifs, average path length), use a fixed sample budget (10^5 samples) and report standard errors.
3. Save signatures to `data/signatures/<kg_name>.json`.
4. Visualize the **real-graph manifold**: PCA / UMAP project all signatures to 2D, highlight families.

This produces (a) empirical priors for the generator (per-block distributions over realistic target values) and (b) a coverage map against which synthetic-graph signatures can be compared.

3.4 The `kgsynth` generator

A **3-stage procedural generator** with a degree-preserving rewiring loop. The package is general-purpose. Sketch of the API:

```
from kgsynth import Signature, Generator

target = Signature.from_config("config.yaml")
G = Generator(target).sample(num_triples=100_000, seed=0)
```

Stage 1 — Schema sampler

Builds the abstract schema: relations, types, and the schema-level statistics that govern what entities will be allowed to do.

Inputs: target signature (blocks A and C).

Procedure:

1. Sample `|R|` (number of relations) from the target.
2. For each relation `r`:

- sample its **frequency weight** `w_r` from `Zipf(s)` where `s` is the target Zipf exponent;
- sample its **cardinality archetype** ∈ {1-1, 1-N, N-1, N-N} from the target archetype mixture;
- sample expected **functionality** and **inverse-functionality**.

3. Sample `|T|` (number of types) and a Zipf for type sizes.
4. Sample the **type–relation table** `P(r | t)` as a low-rank factorization, rank chosen to match the target.
5. Sample the **relation co-occurrence matrix** `M` as a low-rank approximation matching the target top-k singular values.

Output: schema object `S` (relations with their parameters, types, type–relation table, co-occurrence target). Deterministic given the seed.

Stage 2 — Instantiation (CS-aware wiring)

Builds the graph by sampling characteristic sets first, then edges respecting them. This ensures stars (Block D) are pinned correctly without further work.

Inputs: schema `S`, target `|E|`, target `|V|`, target CS distribution.

Procedure:

1. **Sample entity types.** Draw `|V|` entities; assign each a type `t_v ~ Zipf(type-sizes)`.
2. **Sample the CS multiset.** Draw the multiset of characteristic sets that should appear in the graph, given target `|CS|`, CS Zipf, and CS-size distribution. Each CS is itself sampled as a subset of relations weighted by `P(r | t)` for some type `t`, so CSs are schema-consistent.
3. **Assign a CS to each entity** consistent with its type (`CS_v ⊆ {r : P(r | t_v) > 0}`).
4. **Instantiate outgoing edges.** For each entity `v` with `CS_v = {r₁, …, r_k}`:
   - For each `r ∈ CS_v`, sample an **object-multiplicity** from the per-relation, per-archetype distribution (Block B).
   - Draw that many objects from entities whose type is compatible with `r`'s range. Use **preferential attachment** (probability proportional to current in-degree raised to a target-Zipf-controlled exponent) to produce hubs.
5. **Throttle to hit `|E|` exactly** by truncating the lowest-frequency CSs first or by post-hoc pruning.

Output: graph `G_0` satisfying degree marginals, CS marginals, and type-relation marginals — but motif counts and higher-order structure may drift from target.

Important: its clear that all those consteraints will not be satisfied simultaneously. the goal is to somehow get near

them and try to improve the sampler to get this as good as possible.

Stage 3 — Refinement (degree-preserving rewiring)

Adjusts motif counts (cycles, trees, diamonds, longer paths, K4, …) to match the target *without* disturbing the marginals fixed in stages 1 and 2.

just an Idea (can be improved). A **Maslov–Sneppen rewiring move**: pick two edges (s_1, r, o_1) and (s_2, r, o_2) of the same relation r and swap them to (s_1, r, o_2) and (s_2, r, o_1). Two crucial properties:

1. Every entity's per-relation in- and out-degree is preserved exactly. Therefore characteristic sets, CS sizes, the type–relation table, and all degree statistics are **invariant** under this move.
2. Higher-order structure (triangles, 4-cycles, diamonds, longer paths, k-cycles, k-cliques, tree templates) **does change** with each swap.

So stage 3 explores the space of graphs with stage-2 marginals fixed, steering motif counts toward the target.

Procedure (simulated annealing with Metropolis–Hastings):

```
T  ← initial temperature
G  ← G_0
best ← G

for step = 1 .. budget:
    propose swap (e_i, e_j) of same relation r
    if proposed swap violates type-range constraint: reject; continue
    Δ_motif ← change in motif distance from local recompute
    Δ_path  ← change in path-template distance from local recompute
    Δ       ← w_motif · Δ_motif + w_path · Δ_path
    if Δ < 0:
        accept
    else:
        accept with probability exp(-Δ / T)
    if dist(σ_target, σ_current) < dist(σ_target, σ_best):
        best ← G
    T ← cooling_schedule(step)

return best
```

Distance function between two signatures is a weighted sum of per-feature distances:

- KS distance for continuous distributions (degrees, multiplicities, path-template frequencies);

- absolute relative error for scalar counts (triangles, 4-cycles, diamonds, K4);
- KL divergence for categorical distributions (motif-class proportions).

Weights are exposed as hyperparameters and reported in the experimental writeup.

Practical notes:

- Compute motif-count **deltas** locally. A single edge swap changes the triangle count by an amount that depends ONLY on the common neighborhoods of the involved nodes — this is `O(deg)`, far cheaper than recomputing globally, which would quickly become infeasible.
- For 5- and 6-node motifs: maintain a stratified sample of size 10^5, update incrementally on swaps. Full enumeration is intractable.
- Run for a fixed wall-clock budget per generated graph

3.5 Validation

For each generated graph:

1. Re-measure the full signature.
2. Report per-block distance to target.
3. Plot the distance trace over the rewiring loop (convergence diagnostic).

For the population of generated graphs:

- Project all signatures (real + synthetic) to 2D via PCA / UMAP.
- Show that synthetic graphs **cover** the real-graph manifold.
- Show that synthetic graphs **extend** beyond it (controlled extrapolation by perturbing target signatures).

Start with very small graphs first ! so the generation works fast and we can check what parts dont work.

---

4. Phase 2: Scaling Laws Study

We can now use the generator to generate graphs for the scaling law experiments

4.1 Query generation and labeling

For each generated training graph, the student needs ground-truth cardinalities. Workflow:

1. Generate a graph with `kgsynth`.

2. Sample BGP queries using the **DE-TUM RDF subgraph sampler** (https://github.com/DE-TUM/rdf-subgraph-sampler). Cover query shapes (stars, paths, trees, cycles, mixed) and sizes (2–8 triple patterns); aim for ~10K queries per graph balanced over shape × size.
3. Execute queries against QLever to obtain exact cardinalities → training labels.

4.2 Training grid

Single FICE configuration (architecture, hyperparameters) — only the training data varies. Scale the following:

- `N_graphs` — number of distinct training KGs;
- `|E|_total` — total triples across all training KGs.
- thereby sampling from the real world distributions of the characteristics above

4.3 Test sets

Three held-out evaluation sets, in order of increasing distribution shift:

1. **In-distribution synthetic** — graphs from the same `kgsynth` config used in training.
2. **Out-of-distribution synthetic** — graphs generated with target signatures *outside* the training range (different α, different `|R|`, different motif mix). Tests generalization within the synthetic family.
3. **Real KGs** — DBpedia, Wikidata, SWDF, FB15k-237. Tests sim-to-real transfer.

Each test graph is labeled the same way as training graphs (sampler + QLever).

4.4 Metrics and curve fitting

Per `(training-cell, test-set)` combination, report:

- **median Q-error**: geometric mean of `max(pred/true, true/pred)` over the test queries

Scaling curves. For each `(test set, query shape, query size)`, fit

```
Qerror(N) = a · N^{-b} + c
```

once with `N = N_graphs` (fixed `|E|_total`) and once with `N = |E|_total` (fixed `N_graphs`). Report `b` (the scaling exponent) and `c` (the irreducible-error asymptote) with bootstrap confidence intervals.

Headline question. Does the synthetic-trained model's Q-error on real KGs converge toward the in-distribution Q-error as `N` grows? If yes: synthetic-only training transfers. If no: characterize the gap (which shapes, which sizes, which KGs).

5. Results

1. `kgsynth` **package** — pip-installable, documented. Includes:
   - signature measurement on any RDF graph
   - 3-stage generator with the rewiring loop (or an improvement if you find a better way to do this)
   - validation + coverage tooling
   - command-line entry point and Python API
   - tests, examples, README
2. **Real-KG signature dataset** — JSON files with measured signatures for the target real KGs.
3. **Scaling-law experiments** — training-grid runs, plots, fitted exponents.

---

8. References

- Neumann, T., & Moerkotte, G. (2011). *Characteristic Sets: Accurate Cardinality Estimation for RDF Queries with Multiple Joins.* ICDE.
- Maslov, S., & Sneppen, K. (2002). *Specificity and stability in topology of protein networks.* Science.
- Bressan, M., Chierichetti, F., Kumar, R., Leucci, S., & Panconesi, A. (2018). *Motif counting beyond five nodes.* TKDD.
- Tabourier, L., Roth, C., & Cointet, J.-P. (2011). *Generating constrained random graphs using multiple edge switches.* JEA.
- Hestness, J. et al. (2017). *Deep Learning Scaling is Predictable, Empirically.* .
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models.* .
- DE-TUM RDF subgraph sampler: https://github.com/DE-TUM/rdf-subgraph-sampler
- `powerlaw` package: https://github.com/jeffalstott/powerlaw